

安全审计报告 — 文件上传漏洞专项分析

审计目标：v3alpha（脆弱版本）vs v3beta（安全修复版本）

审计日期：2026-07-09

报告范围：文件上传功能及相关安全改进

一、本次重点修复 — 文件上传漏洞

v3alpha 的头像上传功能位于 `/upload` 路由，存在三连环高危漏洞：**任意文件上传**、**目录穿越**、**文件覆盖**。以下逐一分析。

漏洞1：任意文件上传（无类型检查）

漏洞原理

v3alpha 在接收上传文件时，**完全未对文件类型做任何校验**——既不检查扩展名，也不检查 MIME 类型，更不检查文件真实内容。攻击者可以上传任意类型的文件（PHP 脚本、Python 文件、可执行程序等）到服务器上的公开可访问目录 `static/uploads/`，进而实现远程代码执行（RCE）。

漏洞位置

v3alpha/app.py:

```
# 安全漏洞（故意）：使用原始文件名，无扩展名/MIME/内容检查
filename = file.filename
save_path = os.path.join(UPLOAD_FOLDER, filename)
file.save(save_path)
```

攻击方式

- 攻击者登录系统（注册普通用户即可）
- 访问 `/upload` 页面，选择 PHP 一句话木马 `shell.php` 上传
- 文件被保存至 `static/uploads/shell.php`
- 攻击者访问 `http://target/static/uploads/shell.php?cmd=id`，执行任意系统命令

实际验证：v3alpha 的 `static/uploads/` 目录中确实可以成功保存 `shell.php`（内容为 `<?php system($_GET["cmd"]); ?>`）和 `test.py`。

修复方案

v3beta 从四个维度层层设防：

- 扩展名白名单：**仅允许 `jpg, jpeg, png, gif, webp`
- MIME 类型白名单：**仅允许 `image/jpeg, image/png, image/gif, image/webp`
- 文件内容真实性校验：**读取文件二进制头（magic bytes）验证是否为真实图片（Python 3.13 移除了 `imghdr` 模块，v3beta 使用自定义的 magic-byte 检测函数作为回退方案）
- 内存优先验证：**先将文件读入内存验证内容，通过后才写入磁盘，消除 TOCTOU 竞态窗口

修复代码

v3alpha (漏洞代码) :

```
filename = file.filename
save_path = os.path.join(UPLOAD_FOLDER, filename)
file.save(save_path)
```

v3beta (修复代码) :

```
ALLOWED_EXTENSIONS = {"jpg", "jpeg", "png", "gif", "webp"}
ALLOWED_MIME_TYPES = {"image/jpeg", "image/png", "image/gif", "image/webp"}

# 第一层: 扩展名白名单
if not allowed_file(file.filename):
    return error("不支持的文件格式")

# 第二层: MIME 类型校验
if file.content_type not in ALLOWED_MIME_TYPES:
    return error("不支持的文件类型")

# 第三层: 安全文件名
safe_name = secure_filename(file.filename)

# 第四层: 内存中验证文件内容 (避免 TOCTOU)
file_content = file.read()
if not validate_image_content(file_content):
    return error("文件内容不是有效图片")

# 第五层: 验证通过后才写入磁盘
unique_name = f"{uuid4().hex}_{safe_name}"
save_path = os.path.join(UPLOAD_FOLDER, unique_name)
with open(save_path, "wb") as f:
    f.write(file_content)
```

效果验证

场景	v3alpha	v3beta
上传 <code>shell.php</code>	✔ 成功保存, 可直接访问执行	✘ 扩展名白名单拦截
上传 <code>test.py</code>	✔ 成功保存	✘ 扩展名白名单拦截
上传 PHP 内容伪装为 <code>.jpg</code>	✔ 保存成功 (无内容校验)	✘ magic-byte 检测到非图片, 拒绝
上传真实图片	✔ 正常	✔ 通过全部校验

漏洞2：目录穿越

漏洞原理

v3alpha 直接使用用户提供的原始文件名 `file.filename` 构造保存路径，未对路径分隔符（`../`）进行过滤。攻击者可在文件名中插入目录穿越序列，将文件写入 `static/uploads/` 之外的任意路径。

漏洞位置

v3alpha/app.py:

```
filename = file.filename          # 直接使用原始文件名
save_path = os.path.join(UPLOAD_FOLDER, filename) # 未做安全处理
```

攻击方式

构造的文件名	实际写入路径	危害
<code>../../etc/passwd</code>	<code>/etc/passwd</code>	覆盖系统用户数据库
<code>../../var/www/html/backdoor.php</code>	Web 根目录	在任意位置植入 webshell
<code>../../.ssh/authorized_keys</code>	SSH 目录	无密码 SSH 登录

修复方案

`werkzeug.utils.secure_filename()` 自动移除路径分隔符和穿越序列，返回仅含文件名的安全字符串。对于完全由危险字符组成的文件名（如 `../../etc/passwd`），返回空字符串，由代码拒绝上传。

修复代码对比

v3alpha:

```
filename = file.filename
save_path = os.path.join(UPLOAD_FOLDER, filename)
```

v3beta:

```
safe_name = secure_filename(file.filename)
if not safe_name:
    return error("文件名不合法")
unique_name = f"{uuid4().hex}_{safe_name}"
save_path = os.path.join(UPLOAD_FOLDER, unique_name)
```

效果验证

输入文件名	v3alpha	v3beta
<code>../../etc/passwd</code>	写入 <code>/etc/passwd</code>	<code>secure_filename()</code> 返回空串，拒绝
<code>foo/bar.jpg</code>	尝试创建子目录	安全化为 <code>bar.jpg</code>

normal.jpg	static/uploads/normal.jpg	{uuid}_normal.jpg
------------	---------------------------	-------------------

漏洞3：文件覆盖

漏洞原理

v3alpha 不重命名文件，直接使用原始文件名保存。多个用户上传同名文件时后者静默覆盖前者。攻击者可在已知文件名的情况下故意构造同名上传，替换其他用户的头像或文件。

漏洞位置

v3alpha/app.py:

```
filename = file.filename
save_path = os.path.join(UPLOAD_FOLDER, filename)
file.save(save_path) # 直接覆盖，无提示
```

攻击方式

- 1. 用户 A 上传头像 avatar.jpg
- 2. 攻击者观察到文件名，上传同名恶意文件
- 3. 用户 A 的头像被静默替换

修复方案

使用 uuid4().hex 生成 32 字符随机十六进制字符串作为文件名前缀，确保每个文件在文件系统中具有唯一名称。

修复代码对比

v3alpha:

```
save_path = os.path.join(UPLOAD_FOLDER, filename)
```

v3beta:

```
unique_name = f"{uuid4().hex}_{safe_name}"
save_path = os.path.join(UPLOAD_FOLDER, unique_name)
```

效果验证

场景	v3alpha	v3beta
两用户上传同名 avatar.jpg	后者覆盖前者	分别保存为 {uuid1}_avatar.jpg 和 {uuid2}_avatar.jpg
文件枚举	文件名可推测	UUID 随机化，不可预测

二、其他同步发现并修复的问题

白盒审计在 v3beta 开发过程中发现并修复了以下额外问题：

1. Python 3.13 兼容性: imghdr 模块已移除

问题： Python 3.13 从标准库中移除了 `imghdr` 模块。直接 `import imghdr` 会导致 `ModuleNotFoundError`。

修复： 使用 `try/except ImportError` 包裹导入，回退到自定义的 `validate_image_content()` 函数，通过检测文件二进制头（magic bytes）判断真实图片类型：

```
try:
    import imghdr
    def validate_image_content(data):
        return imghdr.what(None, data) in ALLOWED_EXTENSIONS
except ImportError:
    def validate_image_content(data):
        # Magic byte detection fallback for Python 3.13+
        if data[:2] == b'\xff\xd8':          # JPEG
            return True
        if data[:8] == b'\x89PNG\r\n\x1a\n': # PNG
            return True
        if data[:6] in (b'GIF89a', b'GIF87a'): # GIF
            return True
        if data[:4] == b'RIFF' and data[8:12] == b'WEBP': # WebP
            return True
        return False
```

2. MAX_CONTENT_LENGTH 被异常捕获绕过

问题： `validate_csrf()` 中 `except Exception` 捕获了所有异常，包括 Werkzeug 的 `RequestEntityTooLarge` (HTTP 413)，导致超大文件上传时返回误导性的 "CSRF 校验失败" 而非正确的 413 错误。

修复： 在 `except Exception` 前增加 `except RequestEntityTooLarge: raise`，确保 413 异常正确传播到错误处理器。

```
except RequestEntityTooLarge:
    raise
except Exception:
    ...
```

3. 文件先存盘后验证 (TOCTOU 竞态)

问题： 原流程为 `file.save()` → `imghdr.what()` → 不合格则 `os.remove()`。恶意文件在验证完成前已短暂存在于磁盘上。

修复： 改为内存优先验证——`file.read()` 读入内存 → `validate_image_content()` 验证 → 通过后才 `open().write()` 写入磁盘。恶意文件从不会出现在文件系统中。

4. 显式文件大小检查

问题： 仅依赖 `MAX_CONTENT_LENGTH` 限制上传大小，但该限制在 Bug 2 修复前可被绕过。需要深度防御。

修复： 增加 `UPLOAD_MAX_SIZE = 16 * 1024 * 1024` 常量和显式的 `if len(file_content) > UPLOAD_MAX_SIZE` 检查，在内容验证前拦截。

5. 非 ASCII 文件名丢失扩展名

问题： `secure_filename()` 会剥离中文字符，导致 `测试.png` 变成 `png`（无扩展名），Flask 静态文件服务无法返回正确的 Content-Type。

修复： `secure_filename()` 处理后检查是否丢失扩展名，若丢失且原始扩展名在白名单中，重新追加：

```
if "." not in safe_name and original_ext in ALLOWED_EXTENSIONS:
    safe_name = f"{safe_name}.{original_ext}"
```

6. 自定义 413 错误页面

问题： 超过 16MB 的文件上传触发 Werkzeug 默认的英文 413 错误页面。

修复： 添加 `@app.errorhandler(413)` 装饰器，返回用户友好的中文错误提示。

7. CSRF 防护缺失（上传接口）

问题： v3alpha 的 `/upload` 路由未调用 `validate_csrf()`，使上传端点成为全站唯一无 CSRF 防护的 POST 路由。

修复： 在 `/upload` 的 POST 处理入口增加 `validate_csrf()` 校验。

8. 上传目录未自动创建

问题： v3alpha 定义了 `UPLOAD_FOLDER` 但未执行 `os.makedirs()`，首次运行时 `file.save()` 将因目录不存在而崩溃。

修复： 应用启动时执行 `os.makedirs(UPLOAD_FOLDER, exist_ok=True)`。

三、总结

漏洞/问题	严重程度	v3alpha 状态	v3beta 修复方式
任意文件上传（无类型检查）	● 严重	无任何校验	扩展名+MIME+magic-byte 三重校验
目录穿越	● 高危	原始文件名	<code>secure_filename()</code>
文件覆盖	● 中危	不重命名	UUID 前缀
CSRF 防护缺失（上传）	● 高危	有 token 未校验	<code>validate_csrf()</code>
TOCTOU 竞态（先存盘后验证）	● 中危	—	内存验证后写入
MAX_CONTENT_LENGTH 被绕过	● 中危	—	显式 re-raise + 尺寸检查
Python 3.13 imghdr 兼容	● 中危	—	magic-byte 回退方案
非 ASCII 文件名扩展名丢失	● 低危	—	扩展名追加逻辑
413 错误页面未定制	● 低危	—	<code>@app.errorhandler(413)</code>
异常处理缺失	● 低危	无 try/except	完整异常捕获
上传目录未自动创建	● 低危	无 <code>os.makedirs()</code>	启动时创建
冗余 <code>send_from_directory</code>	● 低危	重复路由	移除，使用 <code>url_for("static")</code>

审计完成于 2026-07-09

